Project Proposal: DecodeCache + PIM-BP

Oct 30, 2026 · @NAzmul

We propose a real-time quantum error-correction decoder that **skips work it can** and **makes the remaining work cheaper**. A content-addressable cache answers common error patterns in one lookup; everything else runs belief propagation (BP) inside SRAM, where the messages live, so the algorithm and its answers stay unchanged while data movement disappears. The target is lower tail latency and no syndrome backlog, at the same logical error rate as standard BP.

1. Problem: the decoder must keep pace, and BP is memory-bound

A quantum computer emits a batch of error clues (a syndrome round) roughly every microsecond on superconducting hardware, and a classical decoder must turn each into a correction just as fast. If it falls behind, undecoded rounds pile up, corrections arrive too late for non-Clifford gates, and the computation stalls: the **backlog problem** (preprint).

Two bottlenecks make this hard:

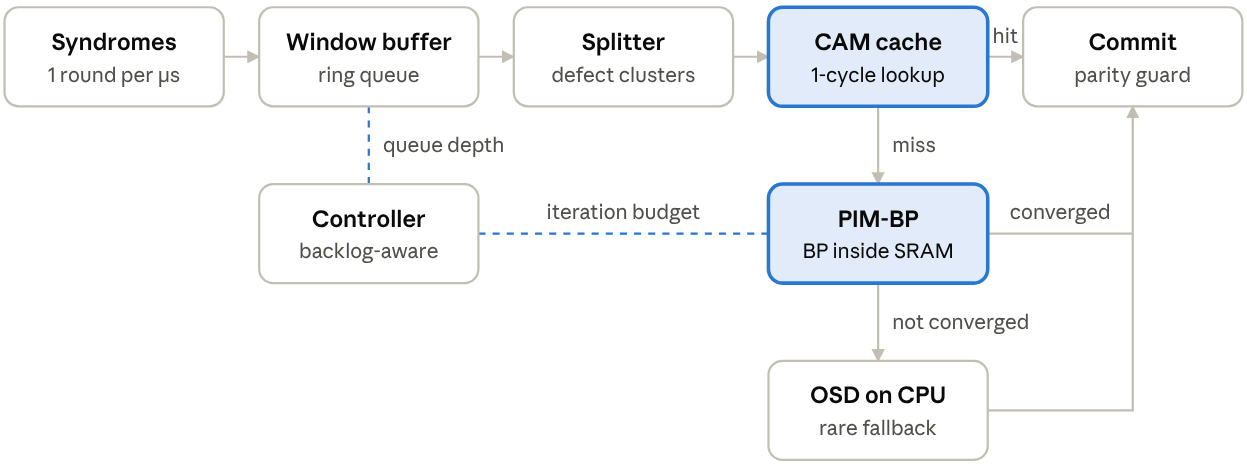
- **The latency tail, not the mean.** Real-time demos usually certify that *mean* decode time is below the round period, but a 2026 analysis argues that bound is the wrong condition; slow outliers can push the queue past a point of no return (preprint). BP's tail comes from windows where messages oscillate and need many more iterations; IBM's Relay-BP was designed to damp exactly this (arXiv).
- **Data movement.** Each BP iteration reads and writes every message on every edge of the Tanner graph, in irregular order, while the math per edge is a single compare or add. On a CPU or GPU, time goes into moving messages, not computing them. The 2026 near-memory BP work cites the same memory-access bottleneck (summary).

The demo that makes this visible to anyone: two decoders with the *same* average speed fed a 1 µs stream; one queue stays flat, the other grows without bound.

2. Proposed system: DecodeCache + PIM-BP

The system has two contributions (highlighted below): a cache that **skips** BP for common patterns, and an in-memory BP engine that makes the remaining BP **cheaper**. A backlog-aware controller ties them together.

Common patterns skip BP; the rest runs BP inside memory



DecodeCache + PIM-BP pipeline · 2 contributions highlighted

Solid arrows carry syndrome data; the dashed loop is control.

Components

1. **Ingest and splitter.** Syndrome rounds enter a ring buffer and are cut into windows; a flood fill groups detection events into connected clusters.
2. **CAM cache tier (DecodeCache).** Each cluster's canonical key (shape translated to the origin, time offsets, boundary flag) is matched in content-addressable memory in one cycle. A hit returns the stored correction immediately. Keys are learned offline from decoded training shots.
3. **PIM-BP engine.** On a miss, normalized min-sum BP runs inside SRAM subarrays that hold the Tanner-graph edge messages:
   - **Check-node update in memory:** sign product via in-array XOR; smallest and second-smallest magnitudes via search-based in-memory minimum finding.
   - **Variable-node update beside the columns:** bit-serial adders sum incoming messages plus the prior and write results back in place.
   - **In-memory convergence check** every iteration: XOR the current guess against the syndrome and OR-reduce to "all zero?". Converged windows exit early.
   - Messages never leave memory between iterations; only the syndrome goes in and the correction comes out.
4. **OSD fallback on CPU.** Ordered-statistics decoding (dense Gaussian elimination) runs only when BP does not converge. It is a poor in-memory fit, so it stays on the host.

5. **Backlog-aware controller.** Watches queue depth and sets BP's iteration budget: generous when the queue is short, tighter when it grows, so slow windows cannot cause a backlog.
6. **Commit with parity guard.** Every correction is checked to reproduce its syndrome before it is applied.

What is new

- Prior near-memory BP work places small processors *beside* DRAM banks (summary); we compute check-node updates *inside* the SRAM arrays.
- Prior predecoders (Clique, Promatch) use fixed rules; we use a learned cluster cache in CAM with a backlog-adaptive controller.
- We evaluate with queueing metrics (tail latency, backlog stability) rather than mean latency alone.

3. Why it is sound

The cache is backed by physics. Below threshold, physical errors are rare ($p \approx 10^{-3}$) and nearly independent, so the probability of a large connected defect cluster falls off exponentially with its size (the percolation argument behind Union-Find and local decoders). Most windows therefore contain a few small clusters drawn from a small set of shapes, which is exactly what a cache serves well.

BP stays exact. We use *digital* in-memory compute: XOR, comparison and addition on quantized messages produce bit-identical results to the same quantized min-sum BP on a CPU. Only where the work happens changes. We avoid analog resistive crossbars, whose device noise would perturb messages and change the algorithm; a design study for classical BP decoders reached the same conclusion in favor of digital SRAM (NSF PAR).

Caveats we will state

- **A cache hit must not change the answer.** A cluster decoded on its own is only correct if it is isolated from other defects and the boundary. The cache-hit condition includes this isolation check; otherwise the cluster goes to BP.
- **Correlated noise** (leakage, cosmic-ray bursts, hook errors) creates large clusters. These become cache misses: they cost latency, not correctness. We test this with injected bursts.

- **Translation invariance is partial** for qLDPC codes and broken near boundaries, so cache hit rates will be lower than for the surface code; we report hit rate per code.
- **Quantization.** Min-sum tolerates few-bit messages well, but we verify the logical error rate at our chosen bit width.
- **Two meanings of "memory".** Relay-BP's memory is an algorithmic damping term, unrelated to hardware memory.
- **Hardware results are modeled**, not measured on silicon.

4. Baselines and metrics

Each baseline isolates one design choice, so every gain can be attributed.

System	What it represents	Isolates
BP+OSD on CPU (`ldpc` package)	Standard software qLDPC decoder	Overall gain
BP on CPU, conventional memory	Same algorithm, data moved to compute	Value of in-memory compute
Near-memory BP on DPUs (2026 paper, published numbers)	Closest prior hardware approach	In-array vs. beside-array compute
IBM Relay-BP on FPGA (published numbers)	State-of-the-art real-time BP hardware	Positioning
PIM-BP without cache	Our engine alone	Value of the cache tier
PIM-BP + cache, no controller	Ablation	Value of the backlog controller
DecodeCache + PIM-BP (full)	Our system	—

For the surface code, also include **PyMatching** (exact matching) as the accuracy reference.

Metrics

- **Logical error rate** vs. physical error rate: must match standard BP within error bars.
- **Latency CDF** with p50 / p99 / p99.9, per window.
- **Backlog over time** at a fixed round period, and the **maximum sustainable round rate**.

- **Cache hit rate** vs. physical error rate, code and distance; **CAM size** vs. hit rate.
- **Data moved per iteration** (bytes) and **modeled energy per decode**.
- **Iterations to convergence** distribution, with and without the in-memory convergence check.

5. Evaluation methodology

We cannot fabricate a chip in a few days, so we split evaluation into a **functional simulation** (is it correct?) and a **performance model** (how fast and how efficient?), then combine them in a **queue simulator** (does the system keep up?).

1. **Workload.** Stim generates circuit-level noisy syndromes for the rotated surface code (d = 5 to 13) and the [[144,12,12]] bivariate-bicycle "gross" code, at p = 0.1% to 0.5%. Stim's detector error model gives the Tanner graph for BP.
2. **Functional simulation.** Python min-sum BP with messages quantized to the chosen bit width, plus the cluster cache. Verify logical error rate against unquantized BP and BP+OSD from the `ldpc` package, using Sinter for the sweeps. Record, per window: cache hit or miss, BP iterations, whether OSD ran.
3. **Performance model.** For each operation in an iteration (edge reads and writes, XORs, comparisons, adds, convergence check), assign latency and energy:
   - *Conventional:* bytes moved across the memory hierarchy at measured or published bandwidth.
   - *PIM-BP:* in-array operation costs from published SRAM compute-in-memory figures, or from a simulator such as NeuroSim or MNSIM 2.0 (MNSIM was used for a prior in-memory QEC decoder study (arXiv)).
   - *CAM:* one lookup per cluster at published CAM search latency.

   Per-window latency = cache path, or (iterations × per-iteration cost) + OSD time if needed.
4. **Queue simulator.** A short discrete-event simulation feeds windows at the round rate and draws service times from step 3. It reports backlog over time, latency CDFs and the maximum sustainable rate, with and without the backlog controller.

Pitfalls to avoid

- Decode the **same shots** with every system (fixed seeds).
- Build the cache from **training** shots and evaluate on a disjoint **test** set.
- Report **tails**, not just means.

- Label every hardware number as **modeled**, and state the sources of per-operation costs.

6. Build plan: 4 days, 3 people

Split by layer so no one blocks anyone.

Day	P1: workload + correctness	P2: cache + PIM-BP model	P3: queue sim + results
1	Stim circuits (surface + gross code); BP+OSD baseline via `ldpc`; save shots	Cluster splitter + canonical key; per-operation cost table from literature	Queue simulator; backlog plot with synthetic heavy-tailed latencies
2	Quantized min-sum BP; LER vs. unquantized (Sinter)	Cache build/test; PIM-BP latency/energy model per iteration	Latency CDF and p99 tooling; plug in real traces
3	Full sweeps; per-window traces (hit, iterations, OSD)	Conventional-memory model for the same BP; controller policy	All plots: LER, CDFs, backlog, max rate, hit rate, bytes moved, energy
4	Reproducibility script; freeze numbers	Live demo script	Slides; two timed rehearsals

- [ ] Day 1: backlog-explodes plot for CPU BP+OSD under a 1 µs round
- [ ] Day 2: quantized BP matches unquantized LER within error bars
- [ ] Day 3: every baseline on every plot
- [ ] Day 4: full rehearsal under 30 minutes

Minimum viable result: cache + quantized BP with the data-movement model (no controller) still gives hit-rate, bytes-moved and latency-CDF comparisons.

7. 30-minute presentation

Open with the backlog explosion so the audience feels the bottleneck before any quantum theory.

Minutes	Segment	Speaker	Content
0–3	Hook	P1	Two decoders, same average speed: one queue flat, one exploding
3–7	Background	P1	Qubits → syndromes → decoder; what BP does in one picture
7–10	Insight 1	P2	Most clusters are small and repeat: cluster-size histogram → "this is a caching problem"
10–13	Insight 2	P2	BP is memory-bound: bytes moved vs. operations per iteration (roofline)
13–18	Architecture	P2	Pipeline diagram: CAM cache, PIM-BP banks, convergence check, OSD fallback, controller
18–25	Results	P3	LER parity; latency CDF; backlog over time; max sustainable rate; hit rate; bytes and energy; ablations
25–27	Demo	P3	Queue simulator: baseline vs. our system on the same stream, side by side
27–30	Limits + Q&A	P1	Modeled not measured; correlated noise; qLDPC hit rates; future FPGA prototype

Record the demo in advance as a backup.

8. Related work and sources

Tools

- Stim and Sinter — syndrome generation, detector error models, LER sweeps
- `ldpc` package (Roffe et al.) — BP and BP+OSD baselines
- PyMatching — matching reference for the surface code
- NeuroSim / MNSIM 2.0 — compute-in-memory cost modeling

Papers

- Backlog dynamics and why mean latency is the wrong bound (preprint)
- Relay-BP, IBM (arXiv)

- Near-memory min-sum BP for the [[144,12,12]] code on DPUs, 2026 (summary)
- Search-based compute-in-memory minimum generation for LDPC min-sum (Beihang)
- SRAM in-memory computing for neural BP channel decoding (NSF PAR)
- Memristive in-memory neural decoder for QEC (arXiv)
- Compute-in-memory FFNN surface-code decoder (arXiv)
- Clique (better-than-worst-case decoding) (abstract) and Promatch (arXiv) — predecoders
- LILLIPUT and AFS — lookup-table and Union-Find decoder hardware (AFS PDF)

Companion docs

- Systems Proposals for Real-Time QEC Decoding
- DecodeCache: BP Compatibility, Physics, and STAR